

- **Lab Title:** Lab 2A: Linux Command Line for Digital Forensics Basics
- **Assignment Title:** Basic Computer Skills for Digital Forensics
- **Student Name:** Bashir Adam
- **Student ID:** 2025/FWSD/11509
- **Course Name:** SBT-DF201: Basic Computer Skills for Digital Forensics
- **Instructor Name:** Aminu Idris
- **Date of Submission:** 15 Aug, 2026
- **Version:** Version 2.1.3

platform used: Kali Linux

Lab Overview

You are a junior digital forensic trainee assigned to prepare a Linux-based evidence workspace. Your task is to demonstrate that you can navigate the Linux filesystem, understand evidence locations, inspect files and metadata, create and manage evidence folders, search for information, check network configuration, write a simple shell script, and preserve your work in a compressed archive.

This lab focuses on Linux command-line skills that are foundational for future forensic work such as disk imaging, file recovery, evidence search, log analysis, and tool automation.

Introduction

This lab focuses on building practical Linux command-line skills that form the foundation for digital forensic work. As a junior forensic trainee, I was tasked with preparing a structured evidence workspace on my Kali Linux system, then using it to practice navigating the filesystem, inspecting file and disk metadata, managing evidence files, searching content with grep, checking network configuration safely, and writing a simple shell script. Linux is widely used in digital forensics because many forensic tools run natively on it, its filesystem follows a consistent, well-documented structure, and its command-line tools give an examiner precise, low-level control over evidence rather than relying only on a graphical interface. This lab connects directly to real forensic tasks such as disk imaging, log analysis, and evidence search, all of which depend on comfort with the same core commands practiced here.

Lab Environment

Linux Distribution: Kali GNU/Linux Rolling (kernel 7.0.12+kali-amd64)

Username: azad

System: Lenovo ThinkPad L440, running natively (not inside a VM) the same physical machine used for Labs 1A and 1B

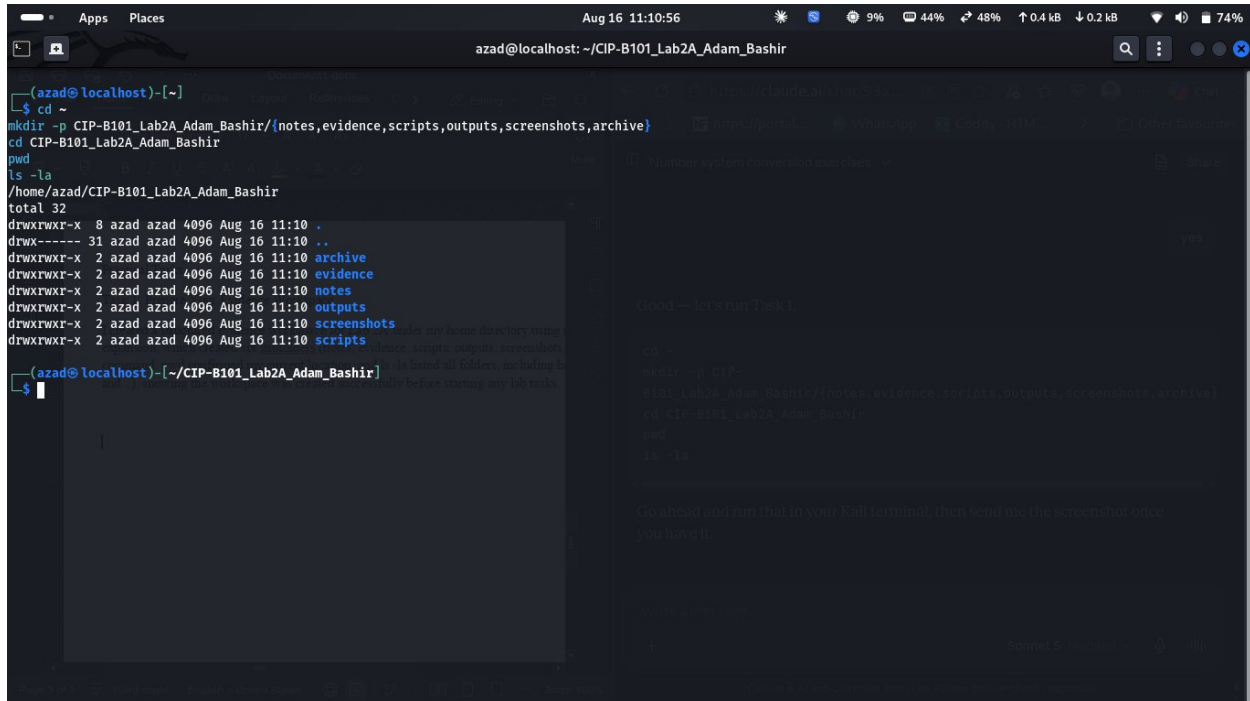
Tools used: Terminal, tree, grep, nano, ss, ping, nmap, zip, tar, and standard core utilities (ls, cd, cp, mv, rm, stat, df, find, readlink)

Internet: Not required for this lab beyond the initial optional package installation

Practical Tasks

Task 1: Prepare Your Evidence Workspace

I created a structured evidence workspace for Lab 2A under my home directory using `mkdir -p` with brace expansion, which created six subfolders (notes, evidence, scripts, outputs, screenshots, archive) in a single command. `pwd` confirmed my current location, and `ls -la` listed all folders, including hidden entries (`.` and `..`), showing the workspace was created successfully before starting any lab tasks.



```
(azad@localhost)-[~]
$ cd ~
mkdir -p CIP-B101_Lab2A_Adam_Bashir/{notes,evidence,scripts,outputs,screenshots,archive}
cd CIP-B101_Lab2A_Adam_Bashir
pwd
/home/azad/CIP-B101_Lab2A_Adam_Bashir
ls -la
total 32
drwxrwxr-x 8 azad azad 4096 Aug 16 11:10 .
drwx----- 31 azad azad 4096 Aug 16 11:10 ..
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 archive
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 evidence
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 notes
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 outputs
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 screenshots
drwxrwxr-x 2 azad azad 4096 Aug 16 11:10 scripts
(azad@localhost)-[~/CIP-B101_Lab2A_Adam_Bashir]
$
```

Task 2: Inspect Linux Filesystem and OS Information

`pwd` confirmed I was still in my lab workspace, `/home/azad/CIP-B101_Lab2A_Adam_Bashir`. `ls /` and `ls -la /` listed the top-level Linux filesystem, showing standard directories like `bin`, `boot`, `dev`, etc, `home`, `lib`, `media`, `mnt`, `opt`, `proc`, `root`, `run`, `sbin`, `srv`, `sys`, `tmp`, `usr`, and `var`. The detailed listing also showed that several entries `bin`, `sbin`, `lib`, `lib64` are actually symbolic links pointing into `usr/` (e.g. `bin -> usr/bin`), which is a modern Linux convention that consolidates binaries under `/usr` while keeping the traditional root-level names for compatibility. I also noticed `/proc` and `/sys` show 0 bytes in size with unusual permissions (`dr-xr-xr-x`) this is expected, since these aren't real files on disk but virtual filesystems generated live by the kernel.

`cat /etc/os-release` confirmed my OS details: Kali GNU/Linux Rolling, version 2026.3, codename kali-rolling, ID=kali, ID_LIKE=debian showing Kali is Debian-based.

```
azad@localhost: ~/CIP-B101_Lab2A_Adam_Bashir
/home/azad/CIP-B101_Lab2A_Adam_Bashir
bin etc initrd.img.old lost+found opt run sys var
boot home lib media proc sbin vmlinuz
dev initrd.img lib64 mnt root srv usr vmlinuz.old
total 76
drwxr-xr-x 19 root root 4096 Jul 23 00:40 .
drwxr-xr-x 19 root root 4096 Jul 23 00:40 ..
lrwxrwxrwx 1 root root 7 Jul 22 15:26 bin -> usr/bin
drwxr-xr-x 4 root root 4096 Aug 9 01:22 boot
drwx----- 2 root root 4096 Jul 22 15:31 .cache
drwxr-xr-x 20 root root 3620 Aug 16 09:04 dev
drwxr-xr-x 144 root root 12288 Aug 16 11:08 etc
drwxr-xr-x 3 root root 4096 Jul 22 15:32 home
lrwxrwxrwx 1 root root 33 Jul 23 00:40 initrd.img -> boot/initrd.img-7.0.12+kali-amd64
lrwxrwxrwx 1 root root 34 Jul 22 15:26 initrd.img.old -> boot/initrd.img-6.12.38+kali-amd64
lrwxrwxrwx 1 root root 7 Jul 22 15:26 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Jul 22 15:26 lib64 -> usr/lib64
drwx----- 2 root root 16384 Jul 22 15:26 lost+found
drwxr-xr-x 4 root root 4096 Jul 22 15:26 media
drwxr-xr-x 2 root root 4096 Jul 22 15:26 mnt
drwxr-xr-x 5 root root 4096 Jul 24 10:56 opt
dr-xr-xr-x 383 root root 0 Aug 11 22:10 proc
drwx----- 8 root root 4096 Aug 10 14:05 root
drwxr-xr-x 38 root root 880 Aug 16 11:08 run
lrwxrwxrwx 1 root root 8 Jul 22 15:26 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Jul 22 15:26 srv
dr-xr-xr-x 13 root root 0 Aug 11 22:10 sys
drwxrwxrwt 18 root root 460 Aug 16 11:12 tmp
drwxr-xr-x 13 root root 4096 Jul 24 10:56 usr
drwxr-xr-x 11 root root 4096 Jul 24 12:44 var
lrwxrwxrwx 1 root root 30 Jul 23 00:40 vmlinuz -> boot/vmlinuz-7.0.12+kali-amd64
lrwxrwxrwx 1 root root 31 Jul 22 15:26 vmlinuz.old -> boot/vmlinuz-6.12.38+kali-amd64
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.3"
VERSION="2026.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
```

Task 3: Understand Inodes and Disk Information

I created a test evidence file, lab2a_evidence.txt, inside my evidence folder using echo with output redirection. ls -li displayed the file's inode number alongside its normal listing details — the inode is the first number shown, before the permissions. stat gave a much more detailed metadata view: file size, block usage, inode number, link count, owning user/group, permission bits, and three separate timestamps (access, modify, and change time). df -h showed disk space usage in human-readable form across all mounted filesystems, while df -i showed inode usage instead of block/space usage — how many inodes are used versus available on each filesystem.

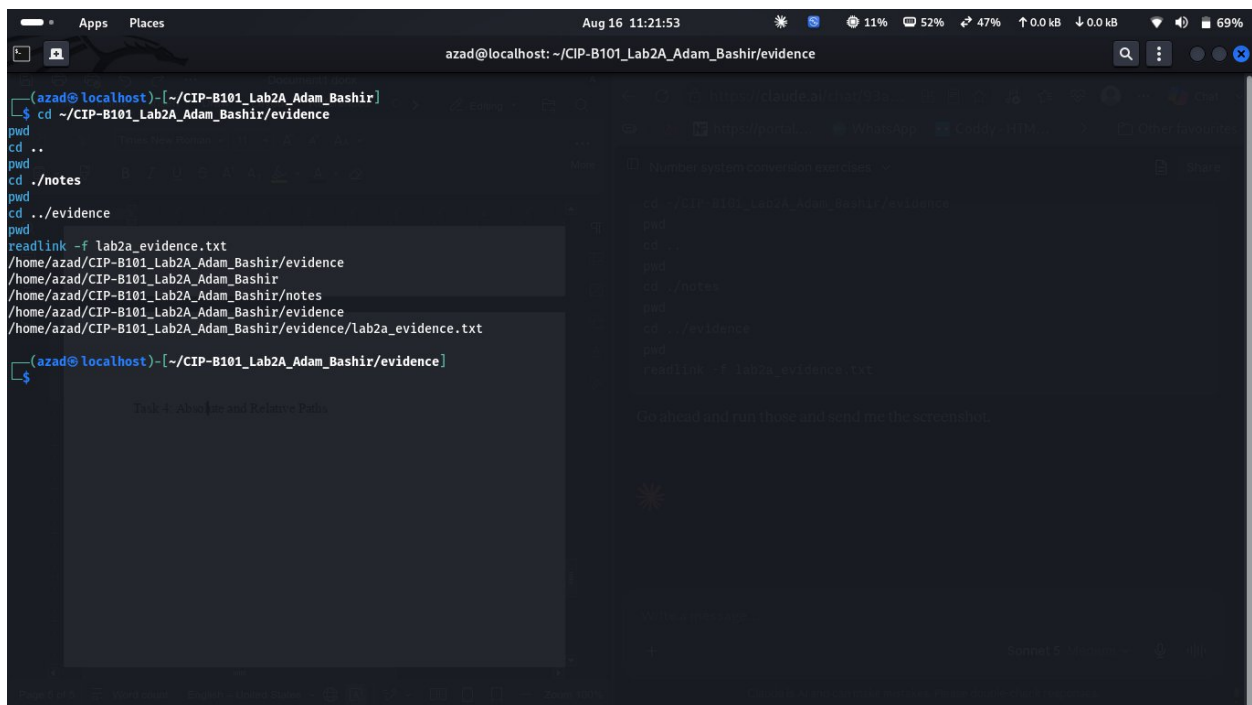
```
azad@localhost: ~/CIP-B101_Lab2A_Adam_Bashir
azad@localhost:~/CIP-B101_Lab2A_Adam_Bashir
$ echo "This is Lab 2A evidence file created by Bashir Adam" > evidence/lab2a_evidence.txt
ls -li evidence/lab2a_evidence.txt
stat evidence/lab2a_evidence.txt
df -h
df -i
19012986 -rw-rw-r-- 1 azad azad 52 Aug 16 11:19 evidence/lab2a_evidence.txt
File: evidence/lab2a_evidence.txt
Size: 52      Blocks: 8      IO Block: 4096   regular file
Device: 8,2  Inode: 19012986  Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  azad)   Gid: ( 1000/  azad)
Access: 2026-08-16 11:19:00.295903597 +0100
Modify: 2026-08-16 11:19:00.295903597 +0100
Change: 2026-08-16 11:19:00.295903597 +0100
Birth: 2026-08-16 11:19:00.295903597 +0100
Filesystem      Size  Used Avail Use% Mounted on
udev            3.7G   0 3.7G   0% /dev
tmpfs           785M   1.7M 783M   1% /run
/dev/sda2       449G   65G 361G  16% /
tmpfs           3.9G  301M  3.6G   8% /dev/shm
efivarfs        72K   28K   40K  42% /sys/firmware/efi/efivars
none            1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
tmpfs           3.9G  110M   3.8G   3% /tmp
/dev/sda1       975M  320K  974M   1% /boot/efi
tmpfs           785M  156K  785M   1% /run/user/1000
Filesystem      Inodes  Used    Ifree  TUse% Mounted on
udev            966988   514   966474    1% /dev
tmpfs           1004176  1013   1003163    1% /run
/dev/sda2       29949952 704096 29245856    3% /
tmpfs           1004176   889   1003287    1% /dev/shm
efivarfs        0         0         0    - /sys/firmware/efi/efivars
none            1024     1     1023    1% /run/credentials/systemd-journald.service
tmpfs           1048576   78  1048498    1% /tmp
/dev/sda1        0         0         0    - /boot/efi
tmpfs           200835   192   200643    1% /run/user/1000
```

Task 4: Absolute and Relative Paths

I used `cd ~/CIP-B101_Lab2A_Adam_Bashir/evidence` an absolute path, since it starts from my home directory using `~` and specifies the full route regardless of where I currently was. `pwd` confirmed I landed at `/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence`.

`cd ..` moved up one level using a relative path, since `..` always means "parent of wherever I currently am" `pwd` confirmed this took me to `/home/azad/CIP-B101_Lab2A_Adam_Bashir`. `cd ./notes` used another relative path (`./` explicitly meaning "starting from the current directory," though it works the same without it) to move into notes `pwd` confirmed `/home/azad/CIP-B101_Lab2A_Adam_Bashir/notes`. Finally, `cd ../evidence` combined "go up one level, then into evidence" again relative landing back at `/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence`.

`readlink -f lab2a_evidence.txt` resolved the file's full absolute path even though I only typed its relative filename: `/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence/lab2a_evidence.txt`. This is useful forensically because it guarantees an unambiguous, complete reference to a file's true location, regardless of the shorthand used to reach it important when documenting exactly where evidence was found, since a relative path alone is meaningless without also stating what directory it was relative to.



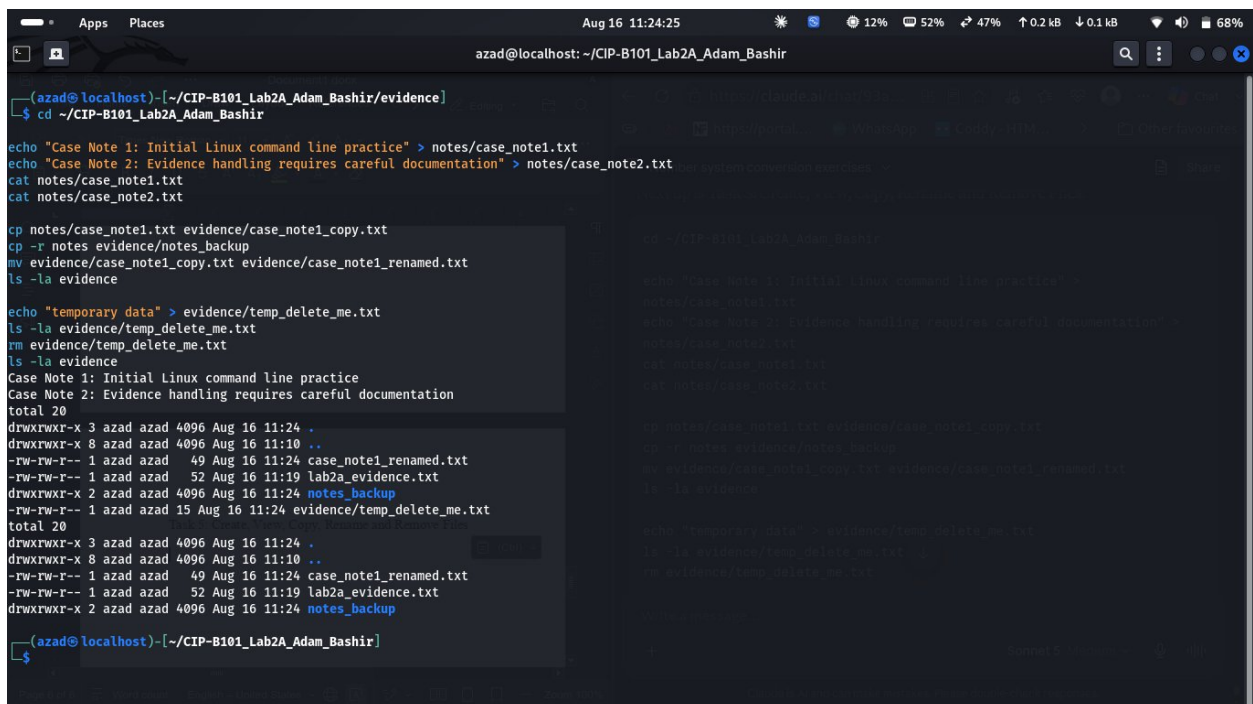
```
(azad@ localhost) [~/CIP-B101_Lab2A_Adam_Bashir]
$ cd ~/CIP-B101_Lab2A_Adam_Bashir/evidence
pwd
/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence
cd ..
pwd
/home/azad/CIP-B101_Lab2A_Adam_Bashir
cd ./notes
pwd
/home/azad/CIP-B101_Lab2A_Adam_Bashir/notes
cd ../evidence
pwd
/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence
readlink -f lab2a_evidence.txt
/home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence/lab2a_evidence.txt
(azad@ localhost) [~/CIP-B101_Lab2A_Adam_Bashir/evidence]
$
```

Task 5: Create, View, Copy, Rename and Remove Files

I created two case note files using echo with > redirection, then confirmed their content with cat: "Case Note 1: Initial Linux command line practice" and "Case Note 2: Evidence handling requires careful documentation."

I then performed several file management operations:

- cp notes/case_note1.txt evidence/case_note1_copy.txt copied case_note1.txt into the evidence folder under a new name.
- cp -r notes evidence/notes_backup recursively copied the entire notes folder (and its contents) into evidence as notes_backup; the -r flag is required for copying directories, since cp alone only works on individual files.
- mv evidence/case_note1_copy.txt evidence/case_note1_renamed.txt renamed the copied file. On Linux, mv is used for both moving and renaming files, since renaming is really just "moving" a file to a new name within the same location.



```
azad@localhost: ~/CIP-B101_Lab2A_Adam_Bashir
(azad@localhost) [~/CIP-B101_Lab2A_Adam_Bashir/evidence]
$ cd ~/CIP-B101_Lab2A_Adam_Bashir

echo "Case Note 1: Initial Linux command line practice" > notes/case_note1.txt
echo "Case Note 2: Evidence handling requires careful documentation" > notes/case_note2.txt
cat notes/case_note1.txt
cat notes/case_note2.txt

cp notes/case_note1.txt evidence/case_note1_copy.txt
cp -r notes evidence/notes_backup
mv evidence/case_note1_copy.txt evidence/case_note1_renamed.txt
ls -la evidence

echo "temporary data" > evidence/temp_delete_me.txt
ls -la evidence/temp_delete_me.txt
rm evidence/temp_delete_me.txt
ls -la evidence

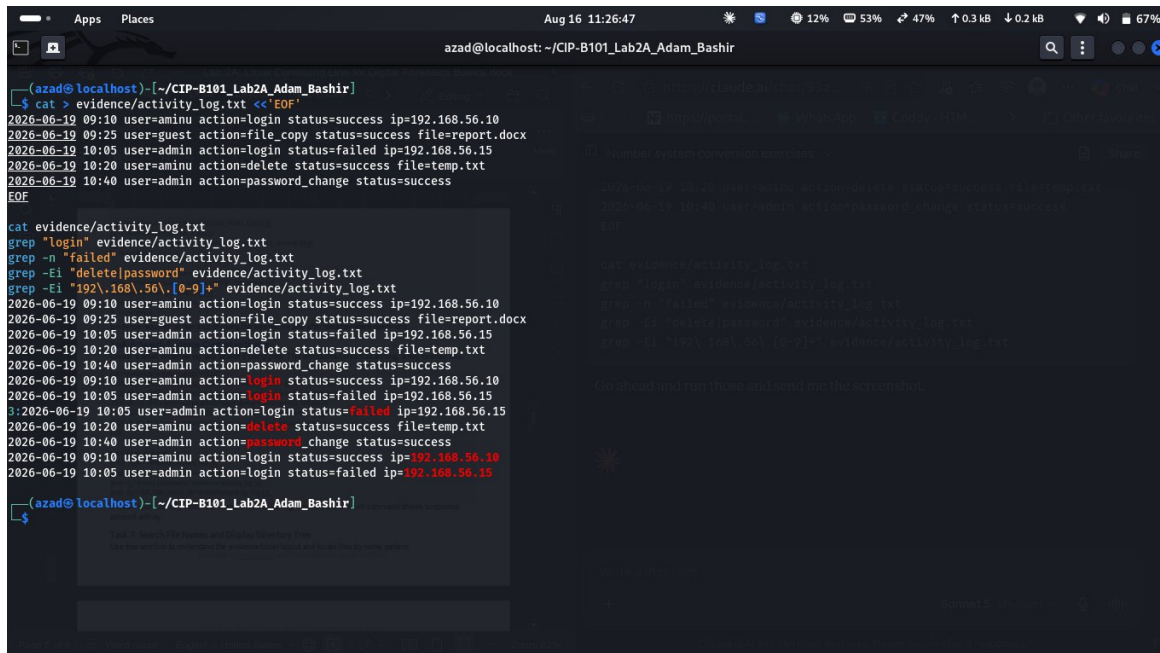
Case Note 1: Initial Linux command line practice
Case Note 2: Evidence handling requires careful documentation
total 20
drwxrwxr-x 3 azad azad 4096 Aug 16 11:24 .
drwxrwxr-x 8 azad azad 4096 Aug 16 11:10 ..
-rw-rw-r-- 1 azad azad 49 Aug 16 11:24 case_note1_renamed.txt
-rw-rw-r-- 1 azad azad 52 Aug 16 11:10 lab2a_evidence.txt
drwxrwxr-x 2 azad azad 4096 Aug 16 11:24 notes_backup
-rw-rw-r-- 1 azad azad 15 Aug 16 11:24 evidence/temp_delete_me.txt
total 20
drwxrwxr-x 3 azad azad 4096 Aug 16 11:24 .
drwxrwxr-x 8 azad azad 4096 Aug 16 11:10 ..
-rw-rw-r-- 1 azad azad 49 Aug 16 11:24 case_note1_renamed.txt
-rw-rw-r-- 1 azad azad 52 Aug 16 11:19 lab2a_evidence.txt
drwxrwxr-x 2 azad azad 4096 Aug 16 11:24 notes_backup

(azad@localhost) [~/CIP-B101_Lab2A_Adam_Bashir]
$
```

Task 6: Search Evidence Content with grep

I created activity_log.txt using a heredoc (cat > file <<'EOF' ... EOF), which let me write multiple lines directly into the file in one block. The log contains five simulated activity entries: two login attempts, a file copy, a delete action, and a password change.

- grep "login" matched every line containing "login" — both the successful login by aminu and the failed login by admin.
- grep -n "failed" matched only the failed login line, and the -n flag prefixed it with its line number in the file: 3:2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15 — pinpointing exactly where in the file that evidence was found.
- grep -Ei "delete|password" used extended regex (-E) with the | (OR) operator to match lines containing either "delete" or "password," case-insensitively (-i). This returned both the delete action by aminu and the password change by admin — two account actions worth reviewing together.
- grep -Ei "192\.168\.56\. [0-9]+" used a regex pattern to match any IP address in the 192.168.56.x range, returning both IP-bearing log lines (the successful login from .10 and the failed login from .15) while ignoring lines with no IP at all.



```
(azad@localhost):~/CIP-B101_Lab2A_Adam_Bashir
$ cat > evidence/activity_log.txt <<'EOF'
2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 09:25 user=guest action=file_copy status=success file=report.docx
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success
EOF

cat evidence/activity_log.txt
grep "login" evidence/activity_log.txt
grep -n "failed" evidence/activity_log.txt
grep -Ei "delete|password" evidence/activity_log.txt
grep -Ei "192\.168\.56\. [0-9]+" evidence/activity_log.txt
2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 09:25 user=guest action=file_copy status=success file=report.docx
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success
2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
3:2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:20 user=aminu action=delete status=success file=temp.txt
2026-06-19 10:40 user=admin action=password_change status=success
2026-06-19 09:10 user=aminu action=login status=success ip=192.168.56.10
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15
(azad@localhost):~/CIP-B101_Lab2A_Adam_Bashir
$
```

which command showed failed login evidence, and which showed suspicious account activity:

grep -n "failed" evidence/activity_log.txt identified the failed login evidence, showing the exact line and line number: admin failed to log in from 192.168.56.15 at 10:05. grep -Ei "delete|password" evidence/activity_log.txt surfaced potentially suspicious account activity by grouping together a file deletion and a password change in the same search — two sensitive actions worth an examiner's closer attention, especially since the password change occurred shortly after the failed login attempt by a different account.

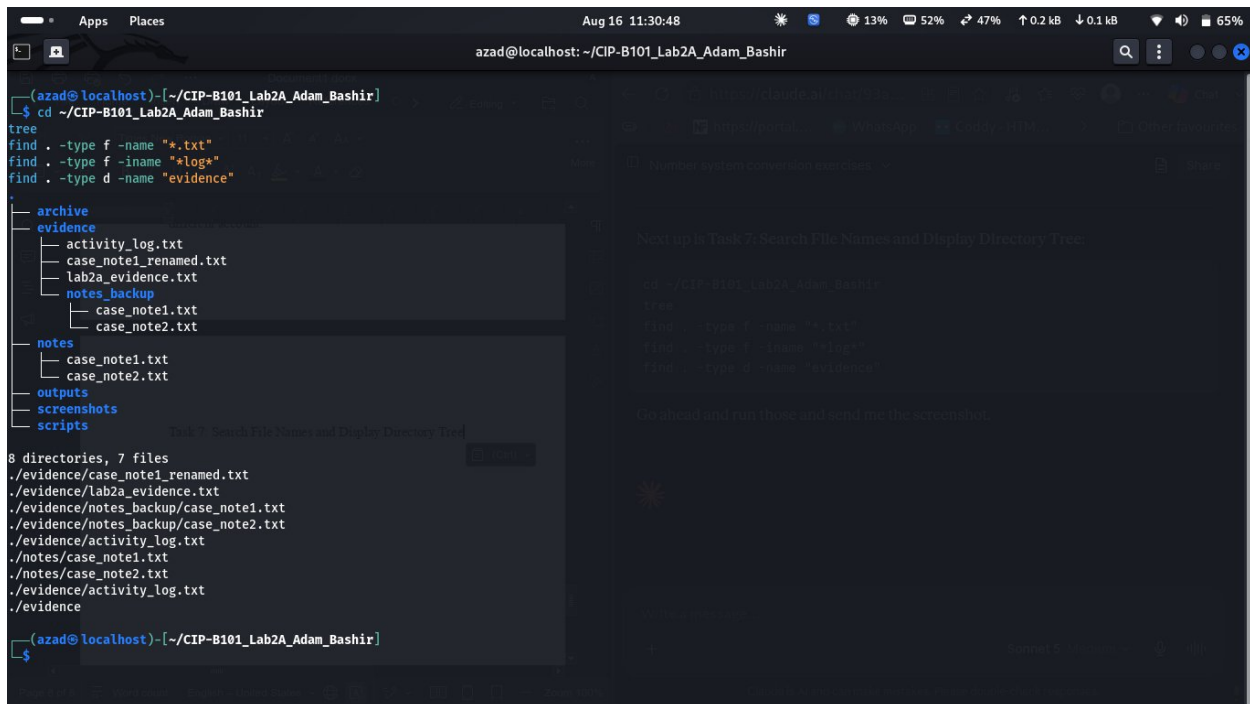
Task 7: Search File Names and Display Directory Tree

tree displayed the complete folder hierarchy of my lab workspace: 8 directories, 7 files. It clearly shows evidence/ containing activity_log.txt, case_note1_renamed.txt, lab2a_evidence.txt, and the notes_backup/ subfolder (holding the recursive copy of both case notes from Task 5), while notes/ holds the two original case note files, and the empty archive, outputs, screenshots, and scripts folders round out the structure — this gives a full visual confirmation that every file operation from earlier tasks landed in the right place.

find . -type f -name "*.txt" searched recursively from the current directory for all regular files (-type f) matching the pattern *.txt, returning 7 results: all my case notes (both originals and the backup copies), lab2a_evidence.txt, and activity_log.txt.

find . -type f -iname "*log*" searched case-insensitively (-iname) for any file with "log" anywhere in its name, correctly isolating just ./evidence/activity_log.txt out of all the files present.

find . -type d -name "evidence" searched specifically for directories (-type d) named exactly "evidence," returning ./evidence confirming find can filter by type (file vs. directory) as well as by name pattern, which is useful when searching evidence for a specific kind of item rather than getting files and folders mixed together in the results.



```
(azad@localhost)~[/CIP-B101_Lab2A_Adam_Bashir]
$ cd ~/CIP-B101_Lab2A_Adam_Bashir
tree
find . -type f -name "*.txt"
find . -type f -iname "*log*"
find . -type d -name "evidence"

.
├── archive
├── evidence
│   ├── activity_log.txt
│   ├── case_note1_renamed.txt
│   ├── lab2a_evidence.txt
│   └── notes_backup
│       ├── case_note1.txt
│       └── case_note2.txt
├── notes
│   ├── case_note1.txt
│   └── case_note2.txt
├── outputs
├── screenshots
└── scripts

Task 7: Search File Names and Display Directory Tree
8 directories, 7 files
./evidence/case_note1_renamed.txt
./evidence/lab2a_evidence.txt
./evidence/notes_backup/case_note1.txt
./evidence/notes_backup/case_note2.txt
./evidence/activity_log.txt
./notes/case_note1.txt
./notes/case_note2.txt
./evidence/activity_log.txt
./evidence

(azad@localhost)~[/CIP-B101_Lab2A_Adam_Bashir]
$
```


Task 8: Check Network Information Safely

ip addr listed my network interfaces: lo (the loopback interface, 127.0.0.1/8), eth0 (a wired interface, currently down/no cable connected state DOWN, NO-CARRIER), and wlan0 (my active Wi-Fi interface, state UP, assigned 192.168.0.105/24). ip route showed my routing table: a default route via 192.168.0.1 through wlan0, and a local route for the 192.168.0.0/24 subnet confirming my Wi-Fi router is my gateway to any outside network.

ping -c 4 127.0.0.1 sent exactly 4 packets (using -c 4 to limit the count instead of pinging forever) to the loopback adLab 2A — Conclusion

This lab gave me hands-on practice with core Linux command-line skills that form the foundation for digital forensic work. Setting up a structured evidence workspace in Task 1 reinforced good habits I've now used across three labs — keeping test data organized and separate from real system files. Inspecting the filesystem hierarchy and OS information in Task 2 helped me understand what folders like /proc, /sys, and /dev actually represent, and connected directly to the "everything is a file" philosophy that makes Linux forensics different from working with Windows.

Working with inodes and disk information in Task 3 was one of the most valuable parts of this lab for me, because it showed that a filename is really just a label — the true identity of a file, from the filesystem's point of view, is its inode. This matters for digital forensics, since it explains why deleted files can sometimes still be recovered, and why an examiner needs to think in terms of inodes and metadata, not just filenames. Practicing absolute and relative paths in Task 4, and file operations like copy, rename, and delete in Task 5, built the same kind of careful, precise habits I need when handling real evidence.

Searching evidence content with grep in Task 6 was directly practical — being able to pull out failed logins, suspicious account actions, and specific IP addresses from a log file with a single command shows exactly why grep is such a core forensic tool. Combining that with tree and find in Task 7 showed how to quickly map out and locate files across a whole evidence folder without checking each one by hand.

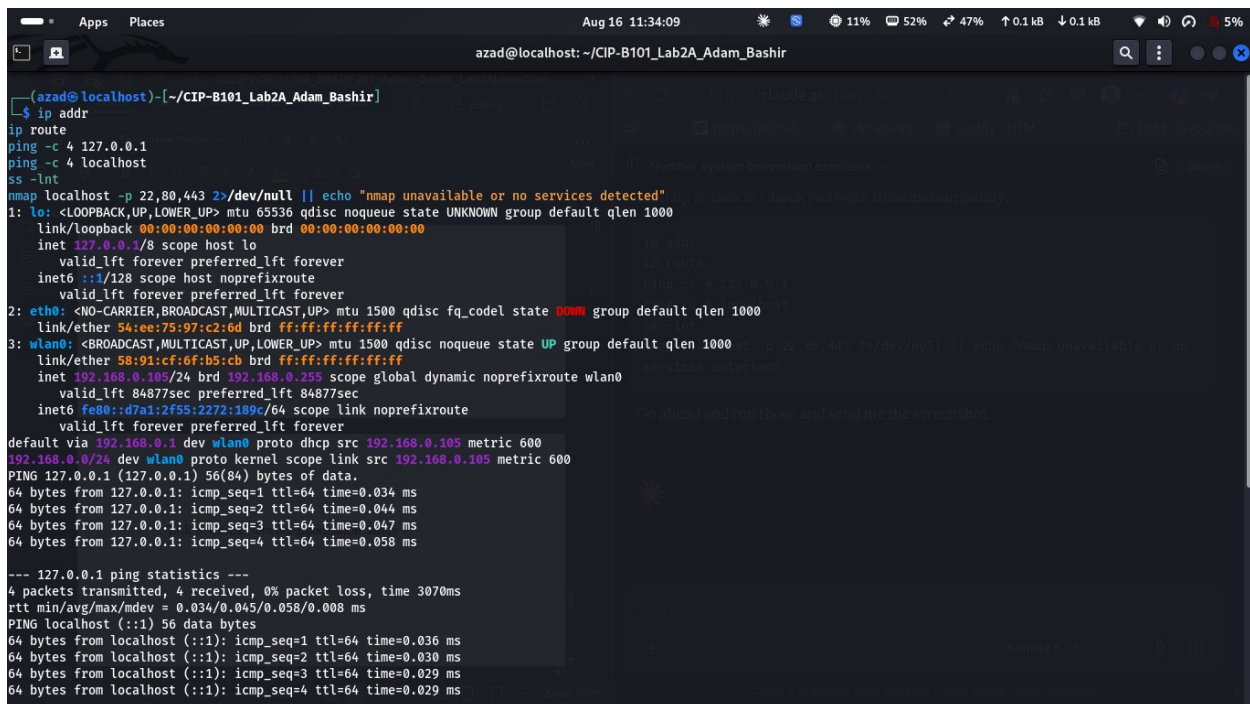
Testing network information safely in Task 8, using only my own loopback address, taught me the important boundary between authorized and unauthorized testing — a boundary I'll need to respect throughout my forensic career. Writing and running my own shell script in Task 9, then making it callable from any folder using PATH in Task 10, tied everything together — and comparing this Linux script's success to the failure I saw with a similar Windows batch script in Lab 1C taught me a deeper lesson: PATH only controls how a command is found and started, but whether a script actually works correctly from any folder depends on whether it uses absolute or relative paths internally.

Finally, compressing my workspace in Task 11 showed a real, measurable difference between zip and tar.gz — not just in theory, but in my own results, where tar.gz came out more than three times smaller than zip for the same files. Overall, this lab strengthened my confidence working entirely from the Linux command line, and gave me tools — path navigation, inode and metadata inspection, grep-based searching, safe network testing, and shell scripting — that I expect to use directly in future, more advanced forensic labs involving disk imaging, log analysis, and file recovery.

That completes the full written content for Lab 2A — Tasks 1 through 12, plus the conclusion. Want me to also do a quick consistency check (like I did for Lab 1A and 1B) once you've got everything compiled into the actual document, or are you good to assemble it yourself from here?dress, with all 4 received, 0% packet loss, and response times around 0.03-0.06ms confirming my own machine's local network stack works. ping -c 4 localhost did the same but resolved "localhost" to the IPv6 loopback ::1 instead, also with 0% packet loss and similarly fast times showing both the IPv4 and IPv6 loopback paths function correctly.

ss -lnt listed listening TCP sockets services currently waiting for incoming connections. My system showed exactly one: a socket listening locally on port 42645, bound to all interfaces (0.0.0.0) a background process on this machine.

nmap localhost -p 22,80,443 scanned my own machine (127.0.0.1) only, checking three common ports: SSH (22), HTTP (80), and HTTPS (443). All three came back closed, meaning no service is currently listening on those specific ports on my system this is a normal, expected result for a machine that isn't running a web server or SSH daemon.



```
azad@localhost: ~/CIP-B101_Lab2A_Adam_Bashir
[azad@localhost]~$ ip addr
ip route
ping -c 4 127.0.0.1
ping -c 4 localhost
ss -lnt
nmap localhost -p 22,80,443 2>/dev/null || echo "nmap unavailable or no services detected"
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether 54:ee:75:97:c2:6d brd ff:ff:ff:ff:ff:ff
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 58:91:cf:6f:b5:cb brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.105/24 brd 192.168.0.255 scope global dynamic noprefixroute wlan0
        valid_lft 84877sec preferred_lft 84877sec
    inet6 fe80::d7a1:2f55:2272:189c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
default via 192.168.0.1 dev wlan0 proto dhcp src 192.168.0.105 metric 600
192.168.0.0/24 dev wlan0 proto kernel scope link src 192.168.0.105 metric 600
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.047 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.058 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3070ms
rtt min/avg/max/mdev = 0.034/0.045/0.058/0.008 ms
PING localhost (::1) 56 data bytes
64 bytes from localhost (::1): icmp_seq=1 ttl=64 time=0.036 ms
64 bytes from localhost (::1): icmp_seq=2 ttl=64 time=0.030 ms
64 bytes from localhost (::1): icmp_seq=3 ttl=64 time=0.029 ms
64 bytes from localhost (::1): icmp_seq=4 ttl=64 time=0.029 ms
```

```
Aug 16 11:34:15
azad@localhost: ~/CIP-B101_Lab2A_Adam_Bashir

inet6 fe80::d7a1:2f55:2272:189c/64 scope link noprefixroute
valid_lft forever preferred_lft forever
default via 192.168.0.1 dev wlan0 proto dhcp src 192.168.0.105 metric 600
192.168.0.0/24 dev wlan0 proto kernel scope link src 192.168.0.105 metric 600
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.047 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.058 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3070ms
rtt min/avg/max/mdev = 0.034/0.045/0.058/0.008 ms
PING localhost (::1) 56 data bytes
64 bytes from localhost (::1): icmp_seq=1 ttl=64 time=0.036 ms
64 bytes from localhost (::1): icmp_seq=2 ttl=64 time=0.030 ms
64 bytes from localhost (::1): icmp_seq=3 ttl=64 time=0.029 ms
64 bytes from localhost (::1): icmp_seq=4 ttl=64 time=0.029 ms

--- localhost ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3062ms
rtt min/avg/max/mdev = 0.029/0.031/0.036/0.003 ms
State Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 0 1024 0.0.0.0:42645 0.0.0.0:*
Starting Nmap ( https://nmap.org ) at 2026-08-16 11:34 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000041s latency).
Other addresses for localhost (not scanned): ::1
PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    closed http
443/tcp   closed https

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds

(azad@localhost)-[~/CIP-B101_Lab2A_Adam_Bashir]
$
```

My loopback address and why localhost testing is safer:

My loopback address is 127.0.0.1 (IPv4) and ::1 (IPv6). Loopback traffic never leaves the machine it's handled entirely internally by the operating system's own network stack, so testing against it cannot affect, probe, or send any traffic to another device on the network or the internet. This makes localhost testing completely safe and legal to practice freely, since there's no possibility of it being mistaken for unauthorized scanning of someone else's system. Scanning an external, unknown system without permission, by contrast, could be interpreted as unauthorized network reconnaissance potentially violating computer misuse laws or an organization's acceptable use policy even if the intent was only educational. This is exactly why the lab restricts scanning to localhost only.

Task 9: Create and Execute a Simple Forensic Script

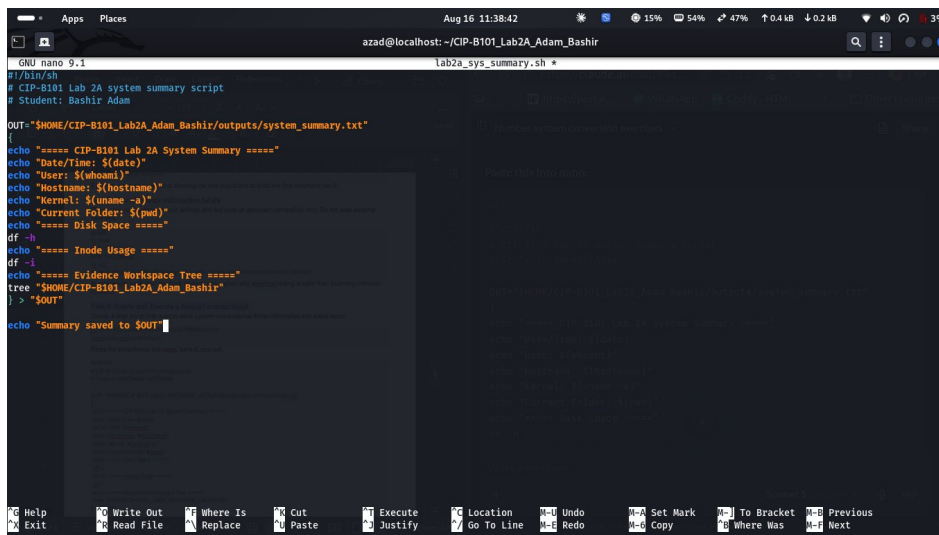
I created lab2a_sys_summary.sh using nano, writing a shell script that gathers system, disk, and workspace information and saves it to a report file. The script uses { ... } > "\$OUT" to group multiple echo and command outputs together and redirect all of them into a single output file in one operation, rather than redirecting each line separately. It captures the date/time, current user, hostname, kernel version, current folder, disk space (df -h), inode usage (df -i), and a full tree of the evidence workspace.

chmod +x lab2a_sys_summary.sh made the script executable, confirmed by ls -l showing the permissions changed to -rwxrwxr-x (note the added x execute bits, compared to a normal text file which would only have read/write). Running ./lab2a_sys_summary.sh executed the script, which printed "Summary saved to /home/azad/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt" confirming it completed successfully.

cat ../outputs/system_summary.txt | head -n 30 displayed the first 30 lines of the generated report, confirming all sections were captured correctly:

- Date/Time: Sun 16 Aug 2026 11:39:30 WAT
- User: azad
- Hostname: localhost
- Kernel: Linux localhost 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64 GNU/Linux
- Current Folder: /home/azad/CIP-B101_Lab2A_Adam_Bashir/scripts
- Disk Space and Inode Usage sections matching the same values I found manually in Task 3
- Evidence Workspace Tree beginning to print at the end of the visible output

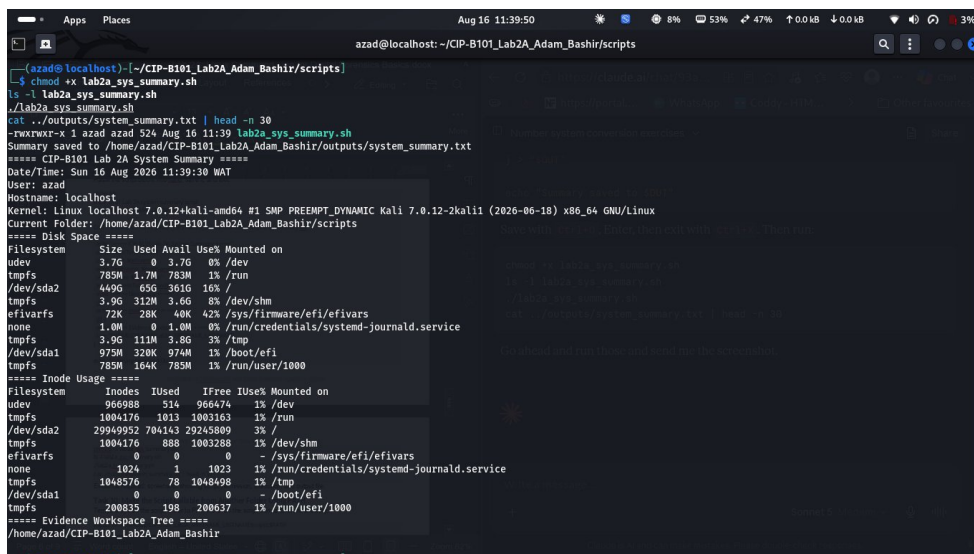
This confirms the script successfully automated collecting the same kind of information I gathered manually in earlier tasks, demonstrating how a single reusable script can save time during real evidence collection.



```
GNU nano 9.1 lab2a_sys_summary.sh
#!/bin/sh
# CIP-B101 Lab 2A system summary script
# Student: Bashir Adam

OUT="$HOME/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt"
{
  echo "===== CIP-B101 Lab 2A System Summary ====="
  echo "Date/Time: $(date)"
  echo "User: $(whoami)"
  echo "Hostname: $(hostname)"
  echo "Kernel: $(uname -a)"
  echo "Current Folder: $(pwd)"
  echo "===== Disk Space ====="
  df -h
  echo "===== Inode Usage ====="
  df -i
  echo "===== Evidence Workspace Tree ====="
  tree "$HOME/CIP-B101_Lab2A_Adam_Bashir"
} > "$OUT"

echo "Summary saved to $OUT"
```



```
azad@localhost:~/CIP-B101_Lab2A_Adam_Bashir/scripts
$ chmod +x lab2a_sys_summary.sh
$ ./lab2a_sys_summary.sh
cat ../outputs/system_summary.txt | head -n 30
-rwxrwxr-x 1 azad azad 524 Aug 16 11:39 lab2a_sys_summary.sh
Summary saved to /home/azad/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt
===== CIP-B101 Lab 2A System Summary =====
Date/Time: Sun 16 Aug 2026 11:39:30 WAT
User: azad
Hostname: localhost
Kernel: Linux localhost 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64 GNU/Linux
Current Folder: /home/azad/CIP-B101_Lab2A_Adam_Bashir/scripts
===== Disk Space =====
Filesystem      Size  Used Avail Use% Mounted on
udev            3.7G     0  3.7G   0% /dev
tmpfs           785M   1.7M  783M   1% /run
/dev/sda2       449G   65G  384G  16% /
tmpfs           3.9G  312M   3.6G   8% /dev/shm
efivarfs        72K    28K   40K  42% /sys/firmware/efi/efivars
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           3.9G  111M   3.8G   3% /tmp
/dev/sda1       975M   528K  974M   1% /boot/efi
tmpfs           785M  164K  783M   1% /run/user/1000
===== Inode Usage =====
Filesystem      Inodes  IUsed  IFree IUse% Mounted on
udev           966988    514 966474   1% /dev
tmpfs          1004176   1013 1003163   1% /run
/dev/sda2     29940932 704143 29236809   3% /
tmpfs         1004176    888 1003288   1% /dev/shm
efivarfs        0         0     0   0% /sys/firmware/efi/efivars
none          1024         1    1023   1% /run/credentials/systemd-journald.service
tmpfs         1048576    78 1048498   1% /tmp
/dev/sda1        0         0     0   0% /boot/efi
tmpfs         200835    198 200637   1% /run/user/1000
===== Evidence Workspace Tree =====
/home/azad/CIP-B101_Lab2A_Adam_Bashir
```

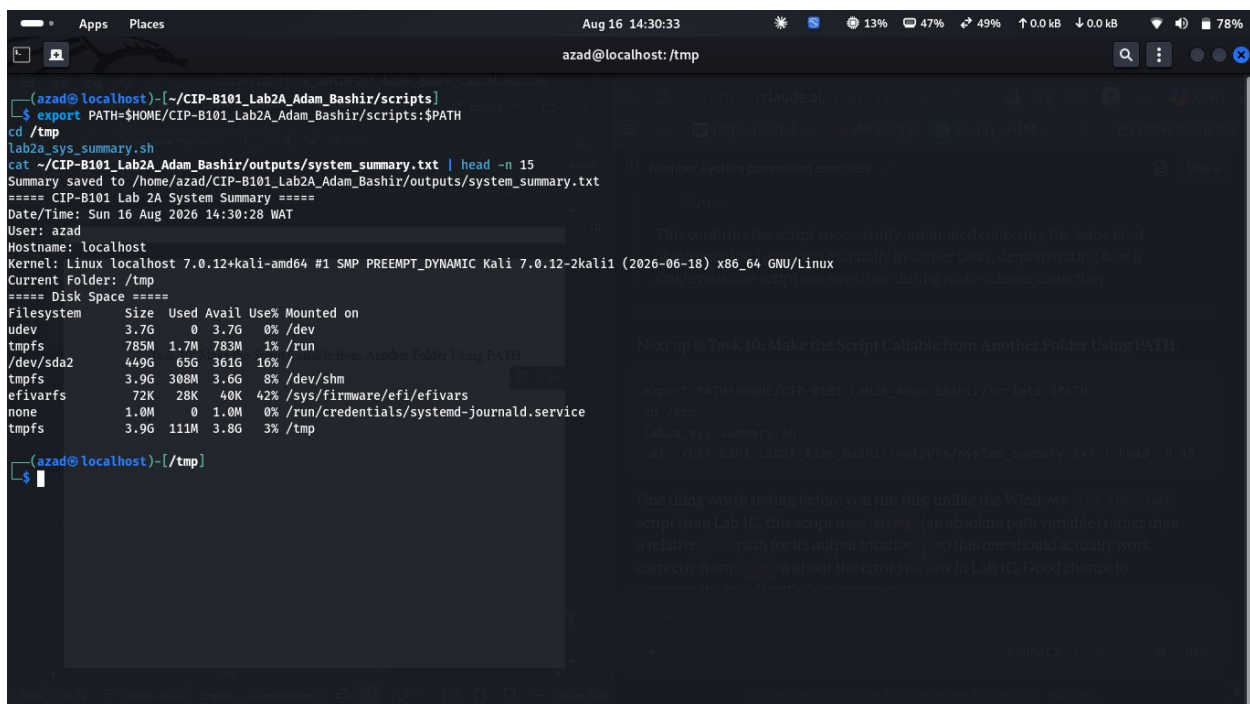
Task 10: Make the Script Callable from Another Folder Using PATH

I temporarily added my scripts folder to PATH using `export PATH=

HOME/CIP-B101Lab2AAdamBashir/scripts:HOME/CIP-B101Lab2AAdamBashir/scripts:PATH'. This adds my scripts folder to the front of the existing PATH list for this terminal session only it does not permanently change PATH, and it will reset once the terminal is closed.

I then moved to a completely different directory, /tmp, using `cd /tmp`, and ran the script by name only `lab2a_sys_summary.sh` with no path in front of it at all. Because /tmp shows Current Folder: /tmp in the output, Linux found and correctly executed the script purely by searching through PATH, without me needing to type its location.

Unlike my Windows batch script in Lab 1C, this script worked perfectly from a completely different folder `cat` confirmed the output file was correctly written to /home/azad/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt, with all sections intact (Date/Time, User, Hostname, Kernel, Current Folder, Disk Space). This is because the script's output path is built using \$HOME an absolute path that always points to /home/azad, no matter what directory the script is run from. This is the exact opposite of the Windows `sys_info.bat` problem in Lab 1C, where the script used a relative path (`..\outputs\...`) that broke as soon as it was run from outside its own folder.



```
azad@localhost: /tmp
(azad@localhost)~[/CIP-B101_Lab2A_Adam_Bashir/scripts]
$ export PATH=$HOME/CIP-B101_Lab2A_Adam_Bashir/scripts:$PATH
cd /tmp
lab2a_sys_summary.sh
cat ~/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt | head -n 15
Summary saved to /home/azad/CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt
===== CIP-B101 Lab 2A System Summary =====
Date/Time: Sun 16 Aug 2026 14:30:28 WAT
User: azad
Hostname: localhost
Kernel: Linux localhost 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64 GNU/Linux
Current Folder: /tmp
===== Disk Space =====
Filesystem      Size  Used Avail Use% Mounted on
udev            3.7G   0  3.7G   0% /dev
tmpfs           785M  1.7M  783M   1% /run
/dev/sda2       449G   65G  361G  16% /
tmpfs           3.9G  308M   3.6G   8% /dev/shm
efivarfs        72K   28K   40K  42% /sys/firmware/efi/efivars
none            1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           3.9G  111M   3.8G   3% /tmp

(azad@localhost)~[/tmp]
$
```

How I'd write it:

The PATH variable is a list of folders that the shell automatically searches, in order, whenever a command is typed by name without specifying its full location. This is exactly why a command like ls can be run from any directory: ls isn't magic it's simply a program file (usually /usr/bin/ls or /bin/ls) sitting in one of the folders already listed in PATH by default. When I typed lab2a_sys_summary.sh from /tmp, the shell searched every folder listed in PATH until it found a matching, executable file which it did, because I had just added my scripts folder to that list. This also directly demonstrates the security point from Lab 1C's PATH task: whichever folder appears first in PATH wins if there are multiple programs with the same name, which is why the order files are added to PATH matters.

Task 11: Compress the Evidence Workspace

I compressed my entire lab workspace two ways for submission. zip -r CIP-B101_Lab2A_Adam_Bashir.zip CIP-B101_Lab2A_Adam_Bashir used the -r flag to recursively include every file and subfolder, showing each item being "added" to the archive including all evidence files, the notes_backup copy, the shell script, and the generated system_summary.txt, with the .txt files showing compression ratios (e.g. activity_log.txt deflated 55%, lab2a_sys_summary.sh deflated 46%) while empty folders and already-small files were simply "stored" with 0% compression.

tar -czvf CIP-B101_Lab2A_Adam_Bashir.tar.gz CIP-B101_Lab2A_Adam_Bashir created a second archive using tar with gzip compression the flags mean create, zip (gzip compress), verbose (list files as they're added), file (specify the output filename). Its verbose output lists the same full folder tree being archived, confirming both methods captured an identical, complete copy of my workspace.

ls -lh confirmed both archives were created successfully:

- CIP-B101_Lab2A_Adam_Bashir.tar.gz 1.7K
- CIP-B101_Lab2A_Adam_Bashir.zip 5.3K

Both archives contain the exact same set of files and folders (confirmed by their matching verbose listings), yet the .tar.gz file is more than 3 times smaller than the .zip file. This is because gzip and zip use different compression approaches: gzip (used by tar.gz) generally achieves stronger compression on a combined stream of many small text files, while zip compresses each file individually within the archive, which adds a small amount of overhead per file and can result in a larger overall archive when there are many small files, as in this workspace.


```
azad@localhost: ~  
$ cd ~  
$ zip -r CIP-B101_Lab2A_Adam_Bashir.zip CIP-B101_Lab2A_Adam_Bashir  
$ tar -czvf CIP-B101_Lab2A_Adam_Bashir.tar.gz CIP-B101_Lab2A_Adam_Bashir  
ls -lh CIP-B101_Lab2A_Adam_Bashir.zip CIP-B101_Lab2A_Adam_Bashir.tar.gz  
adding: CIP-B101_Lab2A_Adam_Bashir/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/case_note1_renamed.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/lab2a_evidence.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/case_note1.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/case_note2.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/evidence/activity_log.txt (deflated 55%)  
adding: CIP-B101_Lab2A_Adam_Bashir/scripts/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/scripts/lab2a_sys_summary.sh (deflated 46%)  
adding: CIP-B101_Lab2A_Adam_Bashir/archive/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/notes/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/notes/case_note1.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/notes/case_note2.txt (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/outputs/ (stored 0%)  
adding: CIP-B101_Lab2A_Adam_Bashir/outputs/system_summary.txt (deflated 58%)  
adding: CIP-B101_Lab2A_Adam_Bashir/screenshots/ (stored 0%)  
CIP-B101_Lab2A_Adam_Bashir/  
CIP-B101_Lab2A_Adam_Bashir/evidence/  
CIP-B101_Lab2A_Adam_Bashir/evidence/case_note1_renamed.txt  
CIP-B101_Lab2A_Adam_Bashir/evidence/lab2a_evidence.txt  
CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/  
CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/case_note1.txt  
CIP-B101_Lab2A_Adam_Bashir/evidence/notes_backup/case_note2.txt  
CIP-B101_Lab2A_Adam_Bashir/evidence/activity_log.txt  
CIP-B101_Lab2A_Adam_Bashir/scripts/  
CIP-B101_Lab2A_Adam_Bashir/scripts/lab2a_sys_summary.sh  
CIP-B101_Lab2A_Adam_Bashir/archive/  
CIP-B101_Lab2A_Adam_Bashir/notes/  
CIP-B101_Lab2A_Adam_Bashir/notes/case_note1.txt  
CIP-B101_Lab2A_Adam_Bashir/notes/case_note2.txt
```

Task 12: Questions to Answer in the Report

1. What is a Linux inode and why is it important?

An inode is a data structure the filesystem uses to store all the metadata about a file its size, permissions, owner, timestamps, and the disk locations where its actual data lives everything except the file's name. In this lab, ls -li showed my evidence file's inode number was 19012986, and stat displayed its full inode-level metadata. Inodes matter forensically because the filename is just a label pointing to an inode; the file's real identity, from the filesystem's point of view, is the inode. This means a file can be renamed or linked under multiple names without changing its underlying data, and when a file is deleted, its inode and data blocks aren't necessarily erased immediately only the directory entry pointing to them is removed, which is the basis for recovering deleted files.

2. What is the difference between an absolute path and a relative path?

An absolute path gives the complete route to a file or folder starting from the root or home directory, and works no matter where you currently are for example, /home/azad/CIP-B101_Lab2A_Adam_Bashir/evidence/lab2a_evidence.txt. A relative path is based on the current directory and only makes sense from that starting point for example, typing evidence/lab2a_evidence.txt while already inside the parent lab folder. In Task 4, I demonstrated this by navigating with cd .. and cd ./notes (relative), then confirming my exact location with readlink -f, which always resolves to the full absolute path.

3. Why does Linux say "everything is a file"? Give two examples from /dev, /proc, /sys, or /etc.

In Linux, hardware devices, running processes, kernel settings, and configuration data are all represented and accessed through the filesystem, using the same file-based interface as a normal document rather than requiring separate, special methods for each type of resource. For example, /dev contains device files representing physical and virtual hardware a disk or terminal can be read from or written to just like a regular file. Similarly, /proc is a virtual filesystem where every running process gets its own numbered folder (like /proc/1234) containing files that expose live information about that process, even though none of it physically exists on disk it's generated by the kernel on demand and presented in file form.

4. What is the purpose of the PATH variable?

PATH is a list of folders that the shell automatically searches, in order, whenever a command is typed by name without giving its full location. This is why a command like ls runs from any folder it isn't built into the shell, it's simply a program file sitting in a folder already listed in PATH. In Task 10, I demonstrated this directly by temporarily adding my scripts folder to PATH with export PATH=..., moving to a completely different directory (/tmp), and running my script by name alone the shell found it by searching through PATH.

5. Which grep command did you use to identify failed login evidence?

I used grep -n "failed" evidence/activity_log.txt, which returned the exact matching line along with its line number in the file: 3:2026-06-19 10:05 user=admin action=login status=failed ip=192.168.56.15. The -n flag was useful because it let me point to precisely where in the file that evidence was located, rather than just knowing it existed somewhere in the log.

6. Why should a forensic trainee avoid scanning unauthorized external systems?

Scanning a system without explicit permission even with harmless intent, like a training exercise can be interpreted as unauthorized network reconnaissance, which may violate computer misuse laws or an organization's acceptable use policy. In Task 8, I limited all testing to my own loopback address (127.0.0.1 and ::1), since loopback traffic never leaves the machine and cannot affect or probe any other device. This kept the exercise both safe and legal, while still practicing the same commands (ping, ss, nmap) an examiner would later use in an authorized, real investigation.

7. What is the difference between zip and tar.gz in this lab context?

zip compresses each file individually within the archive, which makes it easy to extract single files without decompressing the whole archive, and it's natively supported on both Windows and Linux, making it a convenient cross-platform choice. tar.gz first bundles everything into a single uncompressed archive using tar, preserving Linux file permissions and metadata precisely, then compresses that whole bundle as one continuous stream using gzip. In my own test, this made a real difference: my tar.gz file came out to 1.7K, while the zip of the exact same content was 5.3K over three times larger because gzip compresses more efficiently across many small files bundled together, while zip's per-file compression adds overhead for each individual item.

Conclusion

This lab gave me hands-on practice with core Linux command-line skills that form the foundation for digital forensic work. Setting up a structured evidence workspace in Task 1 reinforced good habits I've now used across three labs keeping test data organized and separate from real system files. Inspecting the filesystem hierarchy and OS information in Task 2 helped me understand what folders like `/proc`, `/sys`, and `/dev` actually represent, and connected directly to the "everything is a file" philosophy that makes Linux forensics different from working with Windows.

Working with inodes and disk information in Task 3 was one of the most valuable parts of this lab for me, because it showed that a filename is really just a label the true identity of a file, from the filesystem's point of view, is its inode. This matters for digital forensics, since it explains why deleted files can sometimes still be recovered, and why an examiner needs to think in terms of inodes and metadata, not just filenames. Practicing absolute and relative paths in Task 4, and file operations like copy, rename, and delete in Task 5, built the same kind of careful, precise habits I need when handling real evidence.

Searching evidence content with `grep` in Task 6 was directly practical being able to pull out failed logins, suspicious account actions, and specific IP addresses from a log file with a single command shows exactly why `grep` is such a core forensic tool. Combining that with `tree` and `find` in Task 7 showed how to quickly map out and locate files across a whole evidence folder without checking each one by hand.

Testing network information safely in Task 8, using only my own loopback address, taught me the important boundary between authorized and unauthorized testing a boundary I'll need to respect throughout my forensic career. Writing and running my own shell script in Task 9, then making it callable from any folder using `PATH` in Task 10, tied everything together and comparing this Linux script's success to the failure I saw with a similar Windows batch script in Lab 1C taught me a deeper lesson: `PATH` only controls how a command is found and started, but whether a script actually works correctly from any folder depends on whether it uses absolute or relative paths internally.

Finally, compressing my workspace in Task 11 showed a real, measurable difference between `zip` and `tar.gz` not just in theory, but in my own results, where `tar.gz` came out more than three times smaller than `zip` for the same files. Overall, this lab strengthened my confidence working entirely from the Linux command line, and gave me tools path navigation, inode and metadata inspection, `grep`-based searching, safe network testing, and shell scripting that I expect to use directly in future, more advanced forensic labs involving disk imaging, log analysis, and file recovery.